CineConnect Ticketing System

Software Requirements Specification

Version 1.0

July 13, 2026

Individual Submission

Team Member: Martin Aloka (Red ID: 133942949)

Prepared for

CS 250 – Introduction to Software Systems

Instructor: Gus Hanna, Ph.D.

Summer 2026

Revision History

Date	Description	Author	Comments
July 13, 2026	Version 1.0 – Sections 1–3.3, 3.5	Martin Aloka	First submission for Requirements Specification assignment
July 20, 2026	Version 1.1 – Added Section 6, Software Design Specification	Martin Aloka	Second submission for Software Design Specification assignment
July 27, 2026	Version 1.2 – Added Section 7, Test Plan	Martin Aloka	Third submission for Test Plan assignment

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Martin Aloka	Software Engineer (Student)	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

1. Introduction

The introduction to this Software Requirements Specification (SRS) provides an overview of the complete document. It is intended to contain the information needed by a software engineer to adequately design and implement the CineConnect movie theater ticketing system described herein.

1.1 Purpose

This SRS defines the functional and non-functional requirements for CineConnect, a movie theater ticketing system for a movie theater chain operating twenty (20) theater locations in the San Diego area. The intended audience for this document is the software development team responsible for designing and building the system, the theater chain's management (the client), and the CS 250 course instructional staff evaluating this specification.

1.2 Scope

CineConnect is a browser-based ticketing platform, accessible identically through public web browsers and in-theater self-service kiosks, both of which display the same live-updated data. The system will:

- Display current movies, showtimes, seat availability, and ticket pricing for each of the 20 theaters.
- Allow customers to purchase up to 20 tickets per transaction, with assigned seat selection at deluxe theaters (75 seats) and general-admission tickets at regular theaters (150 seats).
- Support English, Spanish, and Swedish.
- Apply student, military, and senior discount pricing.
- Offer an optional CineConnect Rewards loyalty membership that stores payment information and purchase history and can be used to reserve tickets.
- Accept credit card, PayPal, and cryptocurrency payments, with consistent pricing across all channels.
- Issue unique, non-replicable digital (email) or printable tickets to deter scalping and duplication.
- Collect post-purchase customer feedback and display aggregated critic reviews and scores.
- Provide an administrator console for correcting purchase errors and managing showtimes and theaters.

In its initial release, the system will not be delivered as a native mobile or desktop application (browser access only, per client direction), will not recommend nearby theaters (deferred to a future iteration), and will not automate refunds (refunds continue to be handled by in-person staff).

The system's primary goal is to reduce the transaction friction and downtime that the client's existing legacy ticketing platform suffers from; the client describes the current system as functionally adequate but slow, with bugs that have accumulated over years of incremental updates. The client has requested a viable prototype for user testing within approximately four months and a budget of approximately $500,000.

1.3 Definitions, Acronyms, and Abbreviations

- SRS – Software Requirements Specification
- Deluxe Theater – an auditorium with 75 assigned seats

- Regular Theater – an auditorium with 150 general-admission (unassigned) seats
- CineConnect Rewards – the system's optional customer loyalty membership program
- Kiosk – an in-theater, self-service ticketing terminal running the same web application as the public site
- FR – Functional Requirement
- NFR – Non-Functional Requirement

1.4 References

- Theater Ticketing System Qs.rtf – client interview transcript, collected Week 1, July 8, 2026
- Theater Ticketing Requirements.rtf – client-supplied requirements notes, collected Week 1, July 8, 2026
- theather-ticketing-questions.txt and theather-ticketing-reqs.txt – supplemental client Q&A and requirements notes, collected Week 1, July 8, 2026
- CS 250 – SRS Template, provided by course instructional staff
- IEEE Guide to Software Requirements Specifications, referenced throughout this template

1.5 Overview

The remainder of this SRS is organized as follows: Section 2 describes the product and its intended users at a general level. Section 3 states specific functional and non-functional requirements, including three representative use cases and an accompanying use-case diagram. Sections 4 and 5 and the Appendices are reserved for analysis models, the change-management process, and supplementary materials, which will be developed in a later phase of this project.

2. General Description

This section describes the general factors that affect the product and its requirements. It does not state specific requirements; it makes those requirements, given in Section 3, easier to understand.

2.1 Product Perspective

CineConnect will replace the theater chain's existing legacy ticketing system, which the client considers functionally adequate but slow and increasingly unstable after years of incremental updates. CineConnect is a new, standalone web application — not an extension of the legacy system — that will interface with a single, chain-wide database logically partitioned by theater. The public website and the in-theater kiosks will use the same application and the same live data, so that ticket availability shown in one channel is immediately reflected in the other.

2.2 Product Functions

At a high level, CineConnect will:

- Present showtimes, seat availability, and pricing for each theater.
- Process ticket purchases, including seat selection, discounts, loyalty membership, and payment.
- Issue and validate unique tickets in digital and printable form.
- Support customer account and loyalty membership management.

- Collect customer feedback and display third-party movie review scores.
- Provide administrators with override and showtime/theater management capabilities.
- Maintain a daily, system-wide log of ticket transactions.

2.3 User Characteristics

CineConnect has two primary classes of user:

- Customer – members of the general public purchasing movie tickets, either online from a personal device or in person at a theater kiosk. Customers are assumed to have no specialized training and a wide range of technical proficiency, so the interface must be simple and intuitive; this reflects the client's explicit requirement that “the UI has to be easy to use.” Customers may interact with the system in English, Spanish, or Swedish.
- Administrator – theater chain staff responsible for correcting erroneous customer transactions and for creating, editing, or removing showtimes and theaters. Administrators are assumed to have basic system training and elevated access privileges not available to Customers.

2.4 General Constraints

- The system must be delivered as a browser-based web application; a native mobile or desktop application is explicitly out of scope for this release.
- The system operates within a single time zone (Pacific) and a single metropolitan market (San Diego) for this release.
- The client's target budget for the initial prototype is approximately $500,000, with a four-month timeline to a user-testable prototype.
- All 20 theaters share a single, partitioned database rather than one database per theater.
- Updates to the production system should be deployed as infrequently as practical, since the client has asked that downtime be minimized.

2.5 Assumptions and Dependencies

- The system assumes continued availability of third-party payment processors for credit card, PayPal, and cryptocurrency payments.
- The system assumes continued access to a third-party data source (e.g., Rotten Tomatoes and/or IMDB) for aggregated movie reviews and critic scores; if this access became unavailable, the review-display functionality would need to be revised.
- The system assumes the theater chain remains confined to the San Diego market for the duration of this release; expansion to additional regions or time zones would affect scheduling and scalability requirements.
- The client-provided seat counts (150 regular / 75 deluxe) and showtime cadence (approximately 6–8 showtimes per theater per day) are assumed to remain stable for this release.

3. Specific Requirements

This is the largest and most important section of the SRS. Customer requirements are embodied within Section 2, but this section gives the requirements that guide the project's software design,

implementation, and testing. Each requirement below is intended to be correct, traceable, unambiguous, verifiable, prioritized, complete, consistent, and uniquely identifiable.

3.1 External Interface Requirements

3.1.1 User Interfaces

To be completed in a later phase of this project (user interface requirements).

3.1.2 Hardware Interfaces

To be completed in a later phase of this project (hardware interface requirements).

3.1.3 Software Interfaces

To be completed in a later phase of this project (software interface requirements).

3.1.4 Communications Interfaces

To be completed in a later phase of this project (communications interface requirements).

3.2 Functional Requirements

Functional requirements are organized below into four feature groups. Individual, uniquely identifiable requirements (FR1–FR17) are embedded within the Processing subsection of each feature.

3.2.1 Ticket Search & Purchase

3.2.1.1 Introduction

Allows a Customer, using either the public website or an in-theater kiosk, to find a showtime and complete a ticket purchase.

3.2.1.2 Inputs

- Selected theater and movie
- Desired showtime
- Number of tickets (1–20)
- Seat selection (deluxe theaters only)
- Optional discount type or loyalty membership
- Payment method and payment details

3.2.1.3 Processing

1. FR1 – The system shall display, for both the website and kiosk channels, identical, live-synchronized data on movies, showtimes, ticket prices, and remaining availability.
2. FR2 – The system shall support English, Spanish, and Swedish throughout the purchase flow.
3. FR3 – The system shall allow a Customer to select any showtime within a booking window beginning 2 weeks before and ending 10 minutes after the showtime start.
4. FR4 – The system shall limit a single transaction to a maximum of 20 tickets.

5. FR5 – For deluxe theaters (75 seats), the system shall require selection of specific, assigned seats; for regular theaters (150 seats), the system shall issue general-admission tickets without seat assignment.
6. FR6 – The system shall hold a Customer's in-progress seat/ticket reservation for 5 minutes from the start of checkout, after which the reservation is released if checkout is not completed.
7. FR7 – The system shall apply student, military, or senior discount pricing when selected and eligible, and shall apply the corresponding price consistently regardless of access channel.
8. FR8 – The system shall accept payment by credit card, PayPal, or cryptocurrency.
9. FR9 – Upon successful payment, the system shall generate a unique, non-replicable ticket identifier for each seat/admission purchased.
10. FR10 – The system shall deliver each ticket either as a digital ticket sent by email or as a printable ticket at the box office, per Customer selection.

3.2.1.4 Outputs

- Confirmed reservation with unique ticket identifier(s)
- Digital or printable ticket
- Updated seat/ticket availability visible on all channels
- Transaction record written to the daily transaction log

3.2.1.5 Error Handling

- If payment is declined, the system shall notify the Customer and release the held reservation.
- If the 5-minute checkout window expires before payment completes, the system shall release the reservation and notify the Customer that the session has expired.
- If a requested seat becomes unavailable during checkout, the system shall notify the Customer and prompt an alternate selection.
- If a requested ticket quantity exceeds 20 or the showtime falls outside the booking window, the system shall reject the request with an explanatory message.

3.2.2 Account & Loyalty Management

3.2.2.1 Introduction

Allows a Customer to optionally register for and use a CineConnect Rewards loyalty membership.

3.2.2.2 Inputs

- Customer personal information
- Login credentials
- Preferred payment method
- Membership purchase selection

3.2.2.3 Processing

11. FR11 – The system shall allow a Customer to optionally register a CineConnect Rewards account storing personal information, payment information, purchase history, and loyalty points.
12. FR12 – The system shall allow a registered Customer to purchase and use a loyalty membership to reserve tickets.
13. FR13 – The system shall permit only one device to be concurrently authenticated to a given Customer account at a time.
14. FR14 – The system shall allow a Customer to exchange a ticket prior to the showtime only if it was purchased under a registered account.

3.2.2.4 Outputs

- Confirmed account registration
- Updated purchase history and loyalty point balance
- Login denial notice when a concurrent-device conflict occurs

3.2.2.5 Error Handling

- If a second device attempts to log into an already-authenticated account, the system shall deny the new login and notify the Customer.
- If registration information is incomplete or invalid, the system shall prompt the Customer to correct it before account creation completes.

3.2.3 Administration & Operations

3.2.3.1 Introduction

Allows Administrators to correct customer purchase errors, manage showtimes and theaters, and allows the system to protect itself against abusive/automated purchasing.

3.2.3.2 Inputs

- Administrator login credentials
- Transaction search criteria (e.g., confirmation number)
- Showtime/theater edits
- Incoming purchase request volume

3.2.3.3 Processing

15. FR15 – The system shall provide an administrator mode allowing authorized staff to override erroneous customer purchases, and to create, edit, or remove showtimes and theaters.
16. FR16 – The system shall detect and block automated (bot) purchasing behavior, particularly during high-demand showings.
17. FR17 – The system shall queue and fairly order incoming purchase requests during periods of high demand for a single showing.
18. FR18 – The system shall maintain a system-wide daily log of all ticket purchases.

3.2.3.4 Outputs

- Updated transaction/showtime/theater records
- Daily transaction log entries
- Queue position/wait indication shown to Customers during high-demand periods

3.2.3.5 Error Handling

- If an Administrator attempts to edit a showtime that conflicts with existing reservations, the system shall reject the change and display the conflicting reservations.
- If suspected bot activity is detected, the system shall block or challenge (e.g., CAPTCHA) the requesting client rather than completing the purchase.

3.2.4 Feedback & Content

3.2.4.1 Introduction

Allows Customers to submit post-purchase feedback and view aggregated third-party movie reviews.

3.2.4.2 Inputs

- Customer feedback selection (e.g., smiley-face rating)
- Third-party review/score data for each movie

3.2.4.3 Processing

19. FR19 – The system shall prompt the Customer for feedback (a range of smiley-face ratings) after each completed purchase.
20. FR20 – The system shall display aggregated critic scores and quotes (e.g., from Rotten Tomatoes and/or IMDB) alongside each movie listing.

3.2.4.4 Outputs

- Stored feedback rating
- Displayed critic scores and quotes on the movie listing page

3.2.4.5 Error Handling

- If third-party review data is temporarily unavailable, the system shall display the movie listing without the review panel rather than failing to load.

3.3 Use Cases

The diagram below shows the actors and use cases for CineConnect and the relationships between them. Three representative use cases are then described in detail, following the flow-of-events format introduced in the course lecture notes on Scenarios and Use Cases.

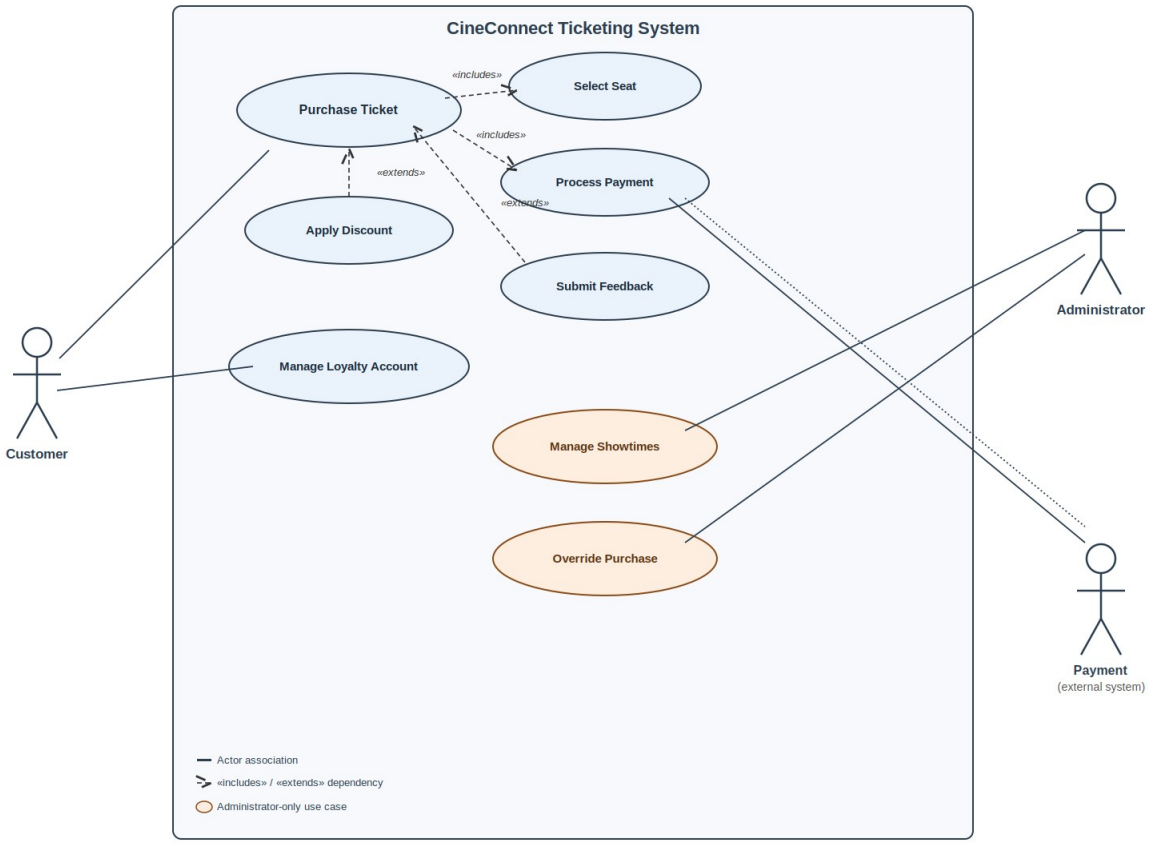

Figure 1. CineConnect use-case diagram.

3.3.1 Use Case #1 – Purchase Ticket

Purpose: describes the use of CineConnect by a representative Customer purchasing movie tickets, from browsing showtimes through receiving a ticket.

Actor(s): Customer

Flow of Events

1. Customer accesses CineConnect via a web browser or an in-theater kiosk.
2. System displays current movies, showtimes, and available seats/pricing for the selected theater.
3. Customer selects a showtime and specifies a ticket quantity (up to 20).
4. If the showtime is at a deluxe theater, Customer selects specific seats from the assigned-seating map; if at a regular theater, System prepares general-admission tickets.
5. Customer optionally applies a discount (student, military, or senior) or a CineConnect Rewards membership.
6. Customer selects a payment method (credit card, PayPal, or cryptocurrency) and completes payment.

7. System generates a unique, non-replicable ticket for each seat/admission purchased and delivers it as a digital ticket by email or a printable ticket.
8. System records the transaction in the daily transaction log.
9. Customer optionally submits post-purchase feedback.

Entry Conditions

1. The selected showtime must fall within its booking window (2 weeks before through 10 minutes after showtime start).
2. The requested ticket quantity must not exceed 20 per transaction.
3. If checkout is not completed within 5 minutes of being started, the reservation is released and the Customer must restart from step 3.

3.3.2 Use Case #2 – Manage Loyalty Account

Purpose: describes how a Customer registers for and uses a CineConnect Rewards loyalty membership.

Actor(s): Customer

Flow of Events

10. Customer selects “My Account” from any access channel.
11. System checks whether the Customer already has a registered account; if not, System prompts the Customer to register.
12. Customer provides personal information, a preferred payment method, and creates login credentials.
13. System creates a CineConnect Rewards account and confirms registration.
14. On a later visit, Customer logs in with their credentials.
15. System verifies that no other device is currently logged into the same account; if one is, System denies the new login and notifies the Customer.
16. Customer views stored payment methods, purchase history, and loyalty points, or uses the account to reserve tickets per the Purchase Ticket use case.
17. Customer logs out.

Entry Conditions

4. Loyalty accounts are optional; a guest Customer may complete Purchase Ticket without one.
5. Only one device may be actively logged into a given account at a time.

3.3.3 Use Case #3 – Override Purchase / Manage Showtimes

Purpose: describes how theater staff correct customer purchasing errors and maintain showtime and theater data.

Actor(s): Administrator

Flow of Events

18. Administrator logs into CineConnect's administrator console using elevated credentials.
19. System displays the administrator dashboard with options to search transactions, manage showtimes, or manage theaters.
20. Administrator searches for a specific customer transaction (e.g., by confirmation number).
21. Administrator selects a correction action: void ticket, reissue ticket, or adjust seat assignment.
22. System applies the override, updates the transaction and daily log accordingly, and notifies the affected Customer if a digital ticket was issued.
23. Alternatively, Administrator selects “Manage Showtimes” and adds, edits, or removes a showtime or theater.
24. System validates that the change does not conflict with existing reservations before committing it.
25. Administrator logs out.

Entry Conditions

6. Administrator must possess valid, elevated (admin-role) credentials.
7. Overrides to a purchase can be applied only prior to the associated showtime.

3.4 Classes / Objects

To be completed in a later phase of this project (class/object modeling).

3.5 Non-Functional Requirements

Non-functional requirements are stated in measurable terms wherever possible.

3.5.1 Performance

- NFR1 – The system shall support at least 1,000 concurrent users across the 20-theater chain in its initial release, with a horizontally scalable architecture capable of growing toward the client's longer-term aspiration of supporting up to 10 million concurrent users if the chain expands beyond San Diego.
- NFR2 – Showtime, seat-map, and pricing pages shall load and refresh within 2 seconds under normal load.
- NFR3 – The system shall queue and fairly order purchase requests during high-demand showings rather than rejecting or losing requests outright.

3.5.2 Reliability

- NFR4 – Ticket prices for a given showtime shall be identical and consistent across the website and kiosk channels at all times.
- NFR5 – Daily transaction logs shall be retained without data loss for audit purposes.

3.5.3 Availability

- NFR6 – The system shall be available 24 hours a day, 7 days a week, excluding scheduled maintenance windows.
- NFR7 – Production updates shall be deployed as infrequently as practical in order to minimize downtime, per client direction.

3.5.4 Security

- NFR8 – Every issued ticket shall carry a unique, non-replicable identifier to prevent duplication, forgery, and resale/scalping.
- NFR9 – No more than one device shall be concurrently authenticated to a given Customer account.
- NFR10 – Payment data shall be handled in accordance with applicable payment card industry (PCI) standards.
- NFR11 – The system shall detect and block automated (bot) purchasing behavior, particularly for high-demand showings.

3.5.5 Maintainability

- NFR12 – The system shall be built with a modular architecture so that individual components (e.g., payment processing, review aggregation) can be updated independently without extended, system-wide downtime.

3.5.6 Portability

- NFR13 – The system shall run entirely within standard, modern web browsers, with no native mobile or desktop application required.
- NFR14 – The same interface and underlying data shall be used for the public website and the in-theater kiosks.

3.6 Inverse Requirements

To be completed in a later phase of this project (inverse requirements).

3.7 Design Constraints

To be completed in a later phase of this project (design constraints).

3.8 Logical Database Requirements

To be completed in a later phase of this project (logical database requirements).

3.9 Other Requirements

To be completed in a later phase of this project (additional requirements).

4. Analysis Models

To be completed in a later phase of this project (analysis models).

4.1 Sequence Diagrams

To be completed in a later phase of this project (sequence diagrams).

4.2 State-Transition Diagrams (STD)

To be completed in a later phase of this project (state-transition diagrams).

4.3 Data Flow Diagrams (DFD)

To be completed in a later phase of this project (data flow diagrams).

5. Change Management Process

To be completed in a later phase of this project (the change management process).

A. Appendices

A.1 Appendix 1 – Client Interview Notes

This appendix is considered informative background and is not itself part of the SRS's normative requirements. The functional and non-functional requirements above were derived from four client-elicitation documents collected during Week 1 (July 8, 2026): a structured client interview (Theater Ticketing System Qs.rtf), a client-supplied requirements list (Theater Ticketing Requirements.rtf), and a supplemental Q&A/requirements pair (theather-ticketing-questions.txt, theather-ticketing-reqs.txt). These source documents are retained on file and are available on request for traceability.

A.2 Appendix 2

To be completed in a later phase of this project (additional appendix material).

6. Software Design Specification

This section is a continuation of the CineConnect Software Requirements Specification above, added as the second deliverable of the course project. Where Sections 1–5 describe what the system must do (intended for the client), this section describes how the system is organized internally for the development team that will implement and maintain it.

6.1 System Description

CineConnect is a browser-based movie ticketing platform for a 20-theater chain based in San Diego, accessible identically through a public website and in-theater kiosks. Customers browse showtimes, reserve seats (at deluxe theaters) or general-admission tickets (at regular theaters), apply discounts or a loyalty membership, and pay by credit card, PayPal, or cryptocurrency, receiving a unique digital or printable ticket. Administrators can override erroneous purchases and manage showtimes and theaters. The system also collects post-purchase feedback and displays aggregated critic reviews. Full functional and non-functional requirements are given in Section 3 above; this section translates those requirements into a concrete software architecture and object model.

6.2 Software Architecture Overview

6.2.1 Architectural Diagram

CineConnect follows a layered, service-oriented architecture: a thin client layer (web and kiosk), an API gateway that handles routing, authentication, and bot mitigation, a set of application microservices that implement the system's core behaviors, a central partitioned database, and a small set of external integrations (payment processors, review data, email delivery, and kiosk printing).

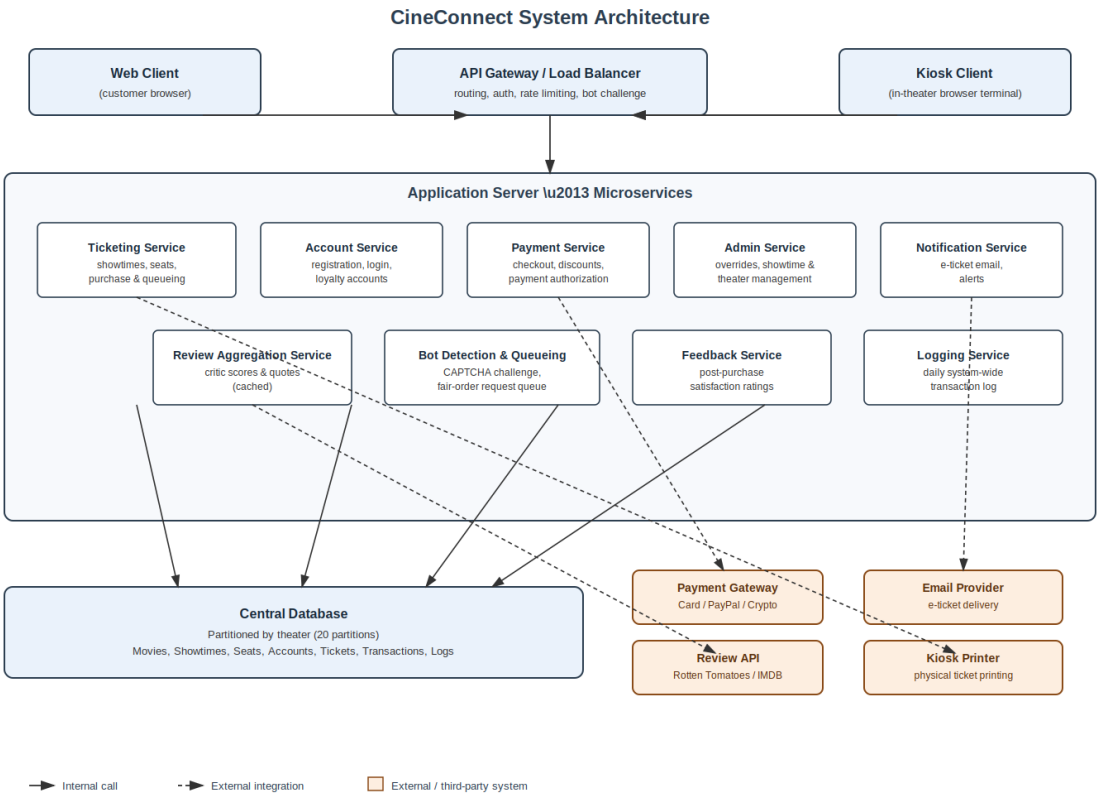

Figure 2. CineConnect system architecture.

6.2.2 UML Class Diagram

The diagram below shows the core domain classes that implement the requirements in Section 3, along with their key attributes, operations, and relationships (association, aggregation, inheritance, and dependency).

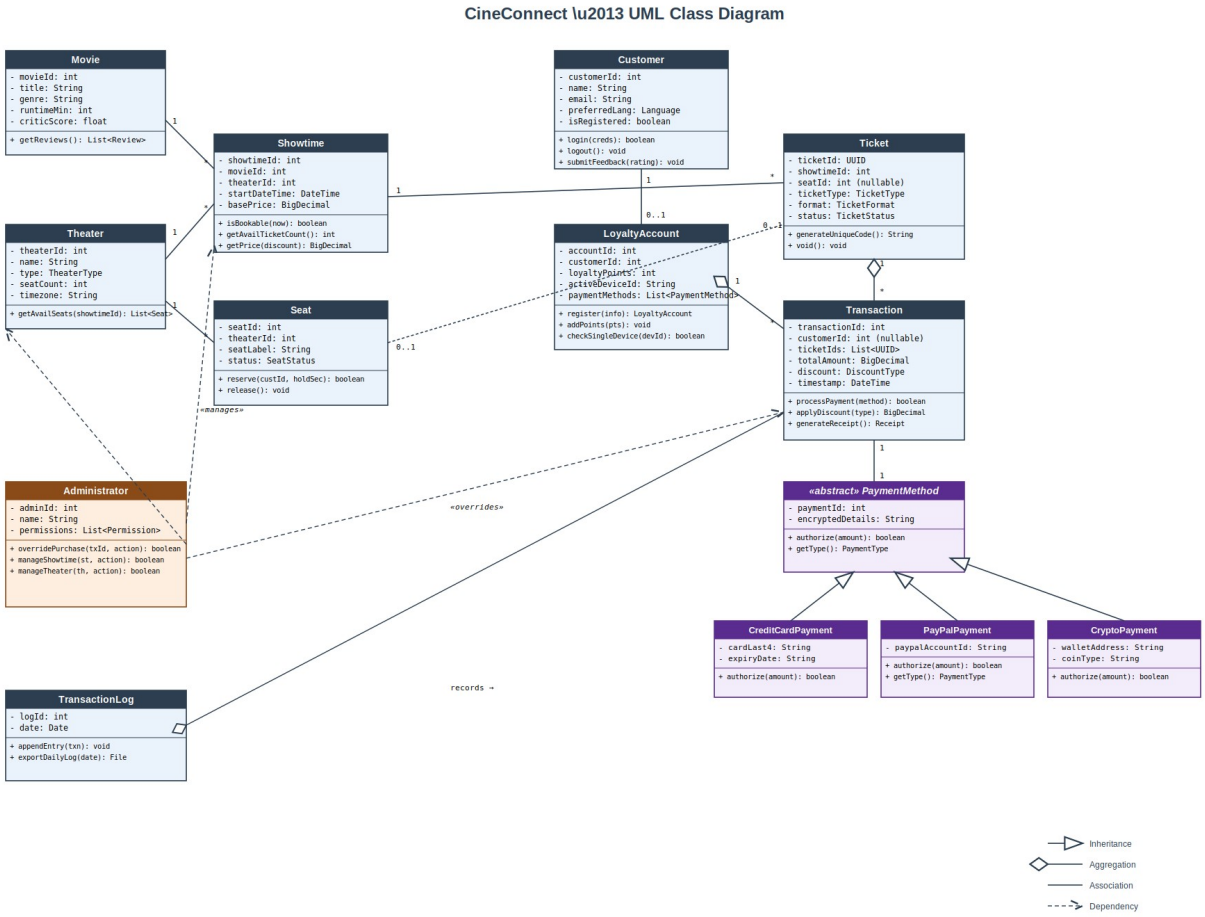

Figure 3. CineConnect UML class diagram.

6.2.3 Description of Classes, Attributes, and Operations

Each class below corresponds to a box in Figure 3. Attribute types and operation signatures are given in full so that they can be implemented directly.

Movie

Represents a film available for showing, along with aggregated third-party critic data.

Attributes

Name	Type	Description
movieId	int	Unique identifier for the movie.
title	String	Movie title.
genre	String	Primary genre classification.

runtimeMin	int	Runtime in minutes.
criticScore	float	Aggregated critic score (e.g., 0–100).

Operations

Operation	Parameters	Returns	Description
getReviews()	none	List<Review>	Retrieves cached critic quotes/scores for the movie (FR20).

Theater

Represents one of the chain's 20 physical theater locations.

Attributes

Name	Type	Description
theaterId	int	Unique identifier for the theater.
name	String	Theater location name.
type	TheaterType {REGULAR, DELUXE}	Determines seating model (150 general-admission vs. 75 assigned seats).
seatCount	int	Total seat capacity.
timezone	String	Local time zone (Pacific for this release).

Operations

Operation	Parameters	Returns	Description
getAvailSeats(showtimeId : int)	showtimeId: int	List<Seat>	Returns seats still available for a given showtime (FR5).

Showtime

Represents a scheduled screening of a Movie at a Theater.

Attributes

Name	Type	Description
showtimeId	int	Unique identifier for the showtime.

movieId	int	Foreign key to Movie.
theaterId	int	Foreign key to Theater.
startDateTime	DateTime	Scheduled start date/time.
basePrice	BigDecimal	Undiscounted ticket price.

Operations

Operation	Parameters	Returns	Description
isBookable(now: DateTime)	now: DateTime	boolean	True if now is within 2 weeks before through 10 minutes after start (FR3).
getAvailTicketCount()	none	int	Returns remaining ticket/seat inventory.
getPrice(discount: DiscountType)	discount: DiscountType	BigDecimal	Returns price after applying an eligible discount (FR7).

Seat

Represents an individual, assignable seat within a deluxe Theater.

Attributes

Name	Type	Description
seatId	int	Unique identifier for the seat.
theaterId	int	Foreign key to Theater.
seatLabel	String	Human-readable seat label (e.g., “D12”).
status	SeatStatus {AVAILABLE, RESERVED, SOLD}	Current seat state.

Operations

Operation	Parameters	Returns	Description
reserve(custId: int, holdSec: int)	custId: int, holdSec: int	boolean	Places a temporary hold on the seat for holdSec seconds (FR6, default 300s).
release()	none	void	Releases a held or cancelled seat back to AVAILABLE.

Customer

Represents any user purchasing tickets, whether or not they hold a LoyaltyAccount.

Attributes

Name	Type	Description
customerId	int	Unique identifier (guest or registered).
name	String	Customer's display name.
email	String	Contact email, used for digital ticket delivery (FR10).
preferredLang	Language {EN, ES, SV}	UI language preference (FR2).
isRegistered	boolean	True if a LoyaltyAccount exists for this customer.

Operations

Operation	Parameters	Returns	Description
login(creds: Credentials)	creds: Credentials	boolean	Authenticates a registered customer.
logout()	none	void	Ends the current session.
submitFeedback(rating: SmileyRating)	rating: SmileyRating	void	Records post-purchase feedback (FR19).

LoyaltyAccount

Represents an optional CineConnect Rewards membership tied to a Customer.

Attributes

Name	Type	Description
accountId	int	Unique identifier for the account.
customerId	int	Foreign key to Customer.
loyaltyPoints	int	Accumulated loyalty points.
activeDeviceId	String	Identifier of the single device currently authenticated (NFR9).
paymentMethods	List<PaymentMethod>	Stored payment methods for faster checkout.

Operations

Operation	Parameters	Returns	Description
register(info: CustomerInfo)	info: CustomerInfo	LoyaltyAccount	Creates a new loyalty account (FR11).
addPoints(pts: int)	pts: int	void	Credits loyalty points after a purchase.
checkSingleDevice(devId: String)	devId: String	boolean	Enforces the single concurrent device rule (FR13).

Ticket

Represents a single, uniquely identifiable admission to one showtime.

Attributes

Name	Type	Description
ticketId	UUID	Globally unique, non-replicable ticket identifier (FR9, NFR8).
showtimeId	int	Foreign key to Showtime.
seatId	int (nullable)	Foreign key to Seat; null for regular-theater general admission.
ticketType	TicketType {REGULAR, STUDENT, MILITARY, SENIOR}	Pricing category applied.
format	TicketFormat {DIGITAL, PRINTED}	Delivery format chosen by the customer (FR10).
status	TicketStatus {ISSUED, USED, VOIDED, REFUNDED}	Current lifecycle state.

Operations

Operation	Parameters	Returns	Description
generateUniqueCode()	none	String	Generates the ticket's non-replicable code at issuance.
void()	none	void	Invalidates the ticket (used by Administrator overrides).

Transaction

Represents a completed or in-progress purchase of one or more Tickets.

Attributes

Name	Type	Description
transactionId	int	Unique identifier for the transaction.
customerId	int (nullable)	Foreign key to Customer; null for guest checkout.
ticketIds	List<UUID>	Tickets included in this transaction (max 20, FR4).
totalAmount	BigDecimal	Final amount charged after discounts.
discount	DiscountType {NONE, STUDENT, MILITARY, SENIOR}	Discount applied, if any.
timestamp	DateTime	Time the transaction was completed.

Operations

Operation	Parameters	Returns	Description
processPayment(method: PaymentMethod)	method: PaymentMethod	boolean	Authorizes and captures payment (FR8).
applyDiscount(type: DiscountType)	type: DiscountType	BigDecimal	Computes the discounted total (FR7).
generateReceipt()	none	Receipt	Produces a receipt/confirmation for the customer.

PaymentMethod (abstract)

Abstract base class for all supported payment types; concrete subclasses implement authorize().

Attributes

Name	Type	Description
paymentId	int	Unique identifier for the stored payment method.
encryptedDetails	String	PCI-compliant encrypted payment details (NFR10).

Operations

Operation	Parameters	Returns	Description
authorize(amount: BigDecimal)	amount: BigDecimal	boolean	Authorizes a charge of the given amount; implemented by each subclass.
getType()	none	PaymentType	Returns CREDIT_CARD, PAYPAL, or CRYPTO.

CreditCardPayment / PayPalPayment / CryptoPayment

Concrete PaymentMethod subclasses corresponding to the three accepted payment types (FR8).

Attributes

Name	Type	Description
cardLast4 / paypalAccountId / walletAddress	String	Type-specific identifying detail, stored in encrypted form.
expiryDate / – / coinType	String	Additional type-specific metadata where applicable.

Operations

Operation	Parameters	Returns	Description
authorize(amount: BigDecimal)	amount: BigDecimal	boolean	Type-specific authorization call to the corresponding external payment gateway.

Administrator

Represents theater-chain staff with elevated permissions.

Attributes

Name	Type	Description
adminId	int	Unique identifier for the administrator.
name	String	Administrator's display name.
permissions	List<Permission>	Set of elevated actions this administrator may perform.

Operations

Operation	Parameters	Returns	Description
overridePurchase(txId: int, action: OverrideAction)	txId: int, action: OverrideAction	boolean	Voids, reissues, or reassigns a seat on an existing transaction (FR15).
manageShowtime(st: Showtime, action: CRUDAction)	st: Showtime, action: CRUDAction	boolean	Creates, edits, or removes a showtime (FR15).
manageTheater(th: Theater, action: CRUDAction)	th: Theater, action: CRUDAction	boolean	Creates, edits, or removes a theater (FR15).

TransactionLog

Represents the system-wide, append-only daily record of ticket purchases.

Attributes

Name	Type	Description
logId	int	Unique identifier for the log entry batch.
date	Date	Calendar date the entries belong to.

Operations

Operation	Parameters	Returns	Description
appendEntry(txn: Transaction)	txn: Transaction	void	Appends a completed transaction to the daily log (FR18).
exportDailyLog(date: Date)	date: Date	File	Exports the full log for a given date for audit purposes (NFR5).

6.3 Development Plan and Timeline

This project is completed by a single-member team (Martin Aloka, Red ID: 133942949). The plan below partitions the remaining implementation work into sequential phases rather than by teammate, and states which requirements/classes from Sections 3 and 6 each phase delivers.

6.3.1 Task Partitioning

Phase	Weeks	Deliverable	Owner
1	1–2	Database schema and core domain classes: Movie, Theater, Showtime, Seat	Martin Aloka
2	3–4	Ticket purchase workflow: Ticket, Transaction, seat holds, discount logic (FR3–FR10)	Martin Aloka
3	5	Account & loyalty workflow: Customer, LoyaltyAccount, single-device login (FR11–FR14)	Martin Aloka
4	6	Payment integration: PaymentMethod hierarchy and gateway integrations (FR8)	Martin Aloka
5	7	Administrator console: overrides, showtime/theater management, daily logging (FR15, FR18)	Martin Aloka
6	8	Bot mitigation, request queueing, feedback, and review aggregation (FR16, FR17, FR19, FR20)	Martin Aloka
7	9	Integration testing against use cases in Section 3.3; bug fixing	Martin Aloka
8	10	Final documentation, deployment prep, and project report	Martin Aloka

6.3.2 Responsibilities

As the sole team member, Martin Aloka is responsible for all design, implementation, and testing tasks above. Each phase is scoped to be completable within its allotted window while still allowing time in later phases for integration and regression testing against requirements delivered in earlier phases.

7. Test Plan

This section is a continuation of the CineConnect SRS, added as the third deliverable of the course project. It lays out the test plan used to verify and validate the design in Section 6 against the requirements in Section 3.

7.1 Software Design Specification (Carried Forward)

The Software Architecture Overview, UML Class Diagram, and class/attribute/operation descriptions from Assignment 2 (Section 6.2 above) remain the authoritative design reference for this test plan; no redesign was necessary; the existing 13-class domain model and 9-component architecture already provide clearly identifiable, independently testable units, features, and system boundaries. Section 7.2 below reuses Figure 3's class structure and Figure 2's architecture to define the scope of each test.

7.2 Test Plan

Testing is organized into three granularities, consistent with the levels discussed in class:

- Unit testing – exercises a single class's method in isolation (dependencies mocked/stubbed), targeting internal logic errors such as boundary conditions, state transitions, and edge cases.
- Functional testing – exercises one service or user-facing feature end-to-end (e.g., the full Purchase Ticket workflow), targeting incorrect feature behavior such as a missed discount or an incomplete override.
- System testing – exercises the complete stack, from client through the application services, database, and external integrations, targeting cross-cutting failures such as cross-channel data inconsistency, performance/scalability limits, and security bypasses (e.g., bots defeating ticket-purchase protections).

The diagram below overlays these three scopes on the architecture from Figure 2, showing which classes and services each granularity of test targets, and mapping each test case ID from Section 7.3 to the component(s) it exercises.

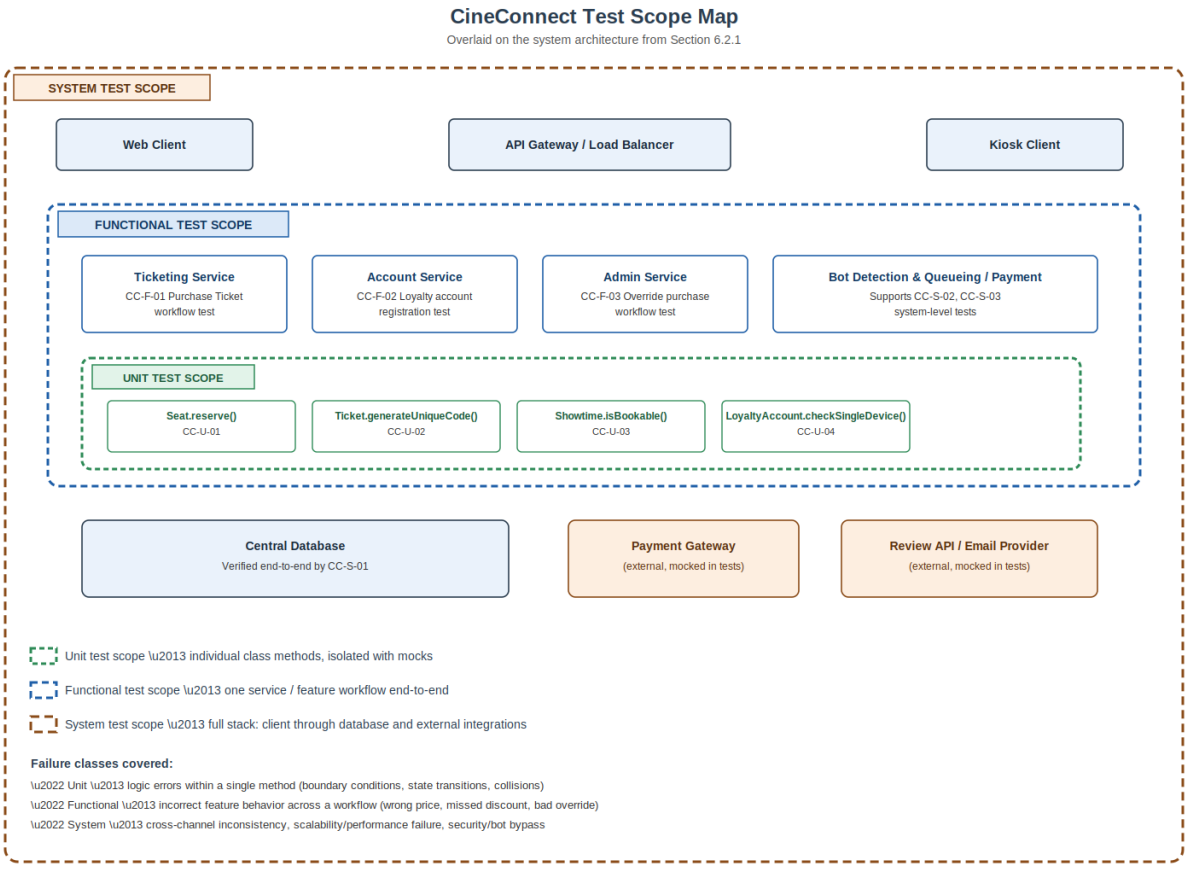

Figure 4. CineConnect test scope map.

Test approach by granularity:

- Unit tests (CC-U-01–CC-U-04) are written against the four domain classes most central to correctness-critical behavior identified in Section 6.2.3: Seat (concurrency/holds), Ticket (uniqueness/anti-fraud), Showtime (booking-window enforcement), and LoyaltyAccount (single-device security rule). Each test instantiates the class directly, without the API gateway, database, or external services, and asserts on return values and resulting object state.
- Functional tests (CC-F-01–CC-F-03) drive one feature end-to-end through its owning microservice (Ticketing, Account, or Admin Service in Figure 2), using a test database and mocked external payment/email endpoints, and assert on the observable outcome of the whole workflow (tickets issued, account created, override applied) rather than on any single class.
- System tests (CC-S-01–CC-S-03) run against a fully deployed staging environment including both client channels, the database, and (where practical) the real or sandboxed external integrations, and assert on cross-cutting, non-functional properties: cross-channel consistency (NFR2), load handling and fair queueing (NFR1, NFR3, FR17), and bot/security resistance (FR16, NFR11).

Coverage note: the ten test cases below satisfy the minimum of two test sets per granularity (4 unit, 3 functional, 3 system) and collectively trace to eleven of the twenty functional requirements and five of the fourteen non-functional requirements defined in Section 3. Additional test cases covering the

remaining requirements (e.g., multi-language support FR2, review aggregation FR20) are planned for a future test cycle beyond this assignment's ten-case minimum.

7.3 Test Case Samples

Ten test case samples are provided in full detail (test steps, pre-requisites, and expected results) in the accompanying Excel workbook, CineConnect_TestCases.xlsx, using the course-provided Test Case Template. A summary is given below; see the workbook for complete step-by-step detail on each case.

ID	Granularity	Component	Priority	Summary
CC-U-01	Unit	Seat	P1	Seat.reserve() places a hold and rejects a concurrent second reservation on the same seat.
CC-U-02	Unit	Ticket	P0	Ticket.generateUniqueCode() never produces a duplicate identifier across 10,000 calls.
CC-U-03	Unit	Showtime	P1	Showtime.isBookable() correctly enforces the 2-week-before / 10-minute-after booking window.
CC-U-04	Unit	LoyaltyAccount	P0	checkSingleDevice() denies a second device while the first remains authenticated.
CC-F-01	Functional	Ticketing Service	P0	End-to-end Purchase Ticket workflow at a deluxe theater with a student discount applied.
CC-F-02	Functional	Account Service	P1	Guest customer registers a loyalty account mid-checkout and completes the purchase under it.
CC-F-03	Functional	Admin Service	P0	Administrator voids an erroneous transaction and the change propagates

				system-wide.
CC-S-01	System	Full System	P0	Web and kiosk clients show identical, live-synced seat availability after a purchase.
CC-S-02	System	Full System / Load	P1	Queueing mechanism fairly processes 1,000 concurrent purchase requests for one showing.
CC-S-03	System	Full System / Security	P0	Simulated bot rapid-purchase script is detected and blocked before ticket issuance.

All ten cases are currently in a planned (“Not Executed”) state, since implementation is scheduled per the development plan in Section 6.3; Actual Result and Test Executed By columns in the workbook will be completed as each case is run during the corresponding implementation phase.

7.4 GitHub Link

The test plan document and the CineConnect_TestCases.xlsx workbook for this assignment have been uploaded to the project's GitHub repository, in the same repository used for the Software Requirements Specification and Software Design Specification deliverables:

https://github.com/martinaloka/cineconnect-ticketing/tree/main/test-plan

Each team member's commit history for this assignment is visible in the repository's commit log at the link above.